

The background features a complex network diagram with numerous nodes of varying sizes (dark blue, light blue, and grey) connected by thin grey lines. Some nodes are highlighted with larger concentric circles. A solid black rectangular box is positioned in the lower right area, containing the title and group information.

AI Agent System Architectures

GROUP 3

AGENDA

- What is an Agent?
- Agents vs. Models
- What is AI Agent Architecture?

From Models to Agents

The Evolution: From Prediction to Action

Generative Models (e.g., GPT-4, Gemini):

- Trained on vast data to predict the next word or pixel.
- They **respond** to prompts.
- Limited to their training knowledge.

Generative Agents:

- Use models as their "brain."
- They **act** to achieve goals.
- Can access external tools and real-time data.
- **Autonomous, Proactive, and Goal-Oriented.**

What is an Agent?



AI agents are autonomous software tools that *perform tasks, make decisions, and interact with their environment intelligently and rationally*.

They can act independently of human intervention, especially *when provided with clear goals*.

They use artificial intelligence to *learn, adapt, and take action based on real-time feedback and changing conditions*.

AI agents *can work on their own or as part of a bigger system*, learning and changing based on the data they process.

Agents vs. Models

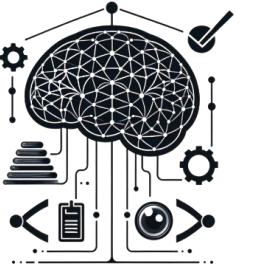
In the rapidly evolving world of artificial intelligence, terms like *"AI agent"* and *"AI model"* are often used interchangeably, leading to confusion. To better understand agents, it's helpful to distinguish them from foundational models:

Aspect	Agent	Model
Knowledge	Agents, however, extend their knowledge by connecting with external systems via tools.	Models are limited to the knowledge available in their training data.
Interaction/Context	Agents, conversely, manage session history (e.g., chat history) to allow for multi-turn inferences and predictions, with decisions informed by the orchestration layer.	Models typically perform a single inference or prediction based on a user query, without native session history or continuous context management
Tool Implementation	Agents, by design, have tools natively implemented within their architecture.	Models do not have native tool implementation
Logic Layer/Cognitive Architecture	Agents possess a native cognitive architecture that uses reasoning frameworks like Chain-of-Thought (CoT), ReAct, or pre-built agent frameworks like LangChain.	Models lack a native logic layer; users guide them with prompts.

What is AI Agent Architecture?

AI Agent Architecture is the **structural blueprint that defines how intelligent software systems perceive, process, and respond to their environment.**

Think of it as the **brain's organization** – just as our neural pathways process information and trigger responses, **AI agent architecture outlines how artificial intelligence programs gather input, make decisions, and take actions.**



Why is It Important to Understand AI Agent Architecture?

Just as our brains process information from our senses to make decisions, **agent architecture provides AI systems with the essential framework to perceive, think, and act.**

- Understanding AI Agent Architecture is crucial for both **technical implementation and business success.**
- It serves as the foundation for building **reliable, scalable, and efficient AI systems** that can deliver **real value in production environments.**

The **right architectural design** can mean the difference between an AI that simply responds to commands and one that truly understands and solves complex problems.



Core Components of Agent Architecture

Sensors (Perception): Collect data from the environment.

Actuators (Action): Execute actions to affect the environment.

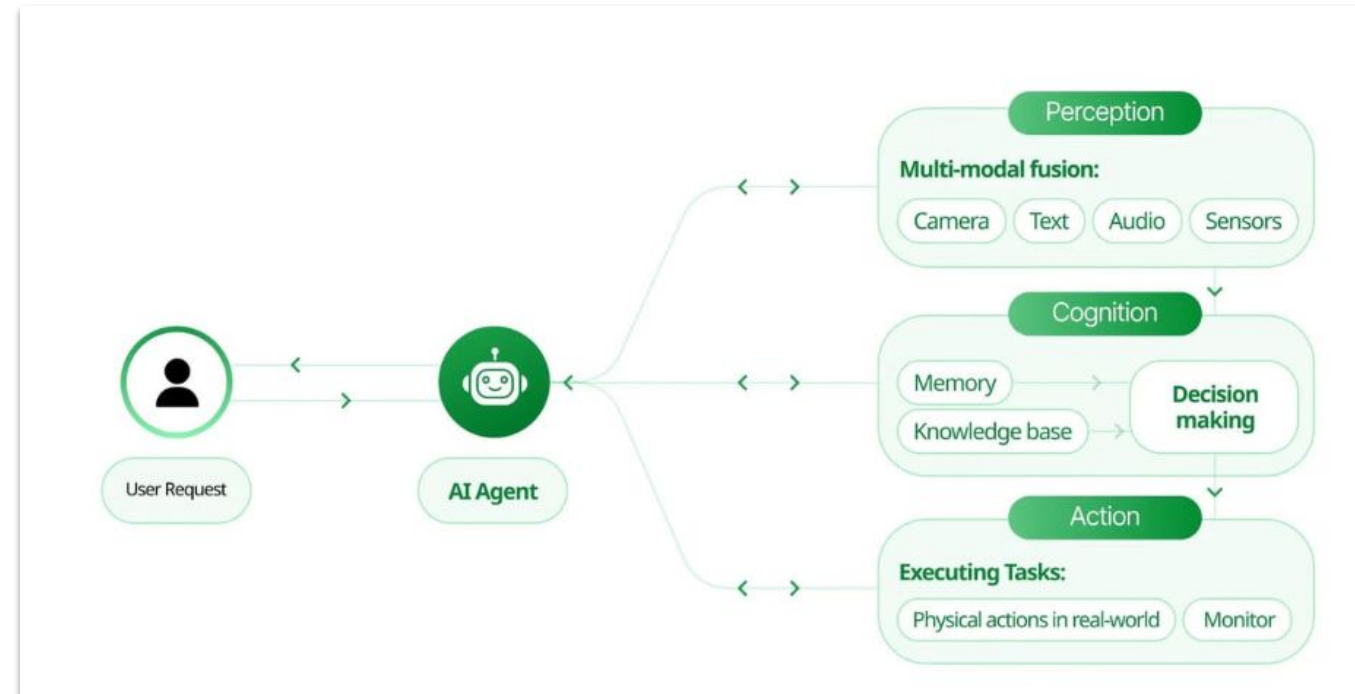
Knowledge Base: Stores facts and learned information.

Reasoning/Decision Module: Chooses actions based on goals and knowledge.

Planning Module: Formulates steps to achieve goals.

Learning Module: Improves decisions based on experience.

Communication Module (optional): Inter-agent communication



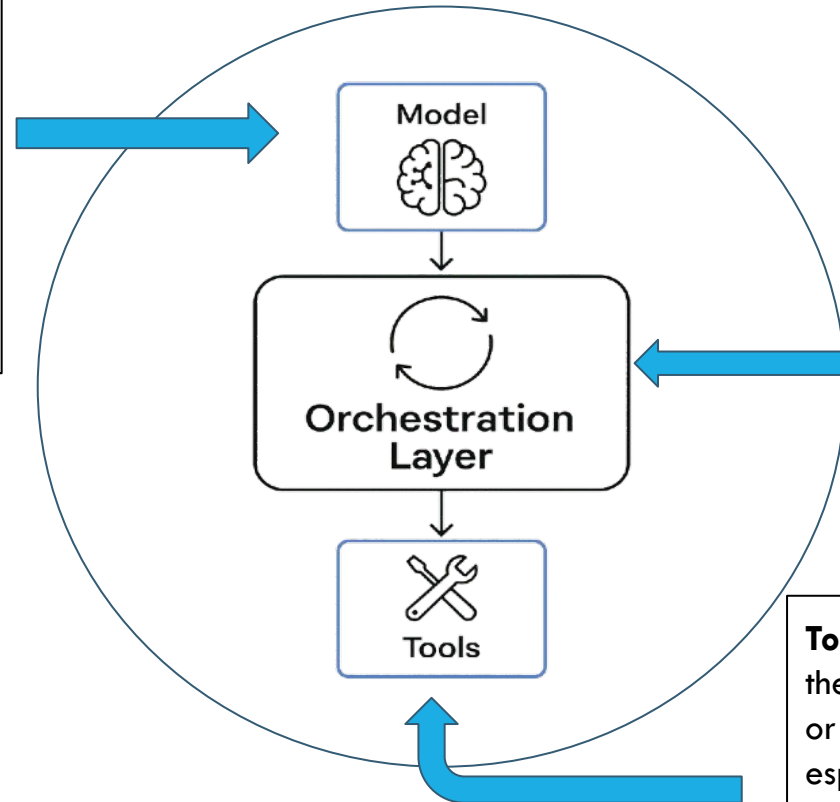
Types of Agent Architectures

- Reactive - Directly maps **sensory input to actions** without internal representations or planning. Suited for **real-time, simple environments**.
- Deliberative - Uses an **explicit internal model** of the environment to reason and **plan before acting**. Capable of **strategic, goal-driven behavior**. Capable of **strategic, goal-driven behavior**.
- Hybrid - Combines **reactive** (fast response) and **deliberative** (planning) components. Balances **real-time responsiveness** with **strategic thinking**.
- BDI (Belief-Desire-Intention) - Based on human-like cognitive structures. Models **rational decision-making** and **goal prioritization**.
- Layered - Separates the agent's behavior into **distinct layers** (e.g., reactive, planning, learning). Promotes **modularity** and **scalability**.
- Factored - Represents the agent's state using **multiple variables/factors** rather than a monolithic state. Useful in **high-dimensional** or **structured environments**.
- Graph-based - Organizes knowledge, goals, or the environment as **nodes and edges** in a graph. Facilitates **relationship modeling**, navigation, and structured reasoning.
- CACA (Collaborative and Adversarial Cognitive Agents)- Agents that **collaborate or compete** using cognitive models of themselves and others. Models **multi-agent dynamics** in realistic, strategic settings.
- Multi Agent Systems - An **integrated system of multiple agents**, possibly using different architectures, working together toward complex tasks. Solves **distributed, multi-perspective problems**.

Core Components of an Agent's Cognitive Architecture

An agent thinks and acts based on core building blocks that make up its "brain" or thinking system (cognitive architecture). While there are many ways to design this system, these **three key parts are essential**.

The Model is the language model (or models) that acts as the **agent's main decision-maker**. It can follow reasoning methods like ReAct or Chain-of-Thought (ToT) and may be general-purpose, fine-tuned, or multimodal—ideally trained to work well with the tools the agent uses.



The Orchestration Layer controls how the agent thinks and acts in a loop—taking in information, reasoning about it, then deciding what to do next. This cycle repeats until the task is done. It can be simple or complex and is in charge of managing memory, logic, and planning using prompt-based guidance.

Tools let the **agent interact with the outside world**—doing things the model alone can't, like getting live data, updating databases, or sending emails. They greatly expand what the agent can do, especially in systems like Retrieval-Augmented Generation (RAG). For example, Google models use tools like **Extensions, Functions, and Data Stores**.

The Power of Teamwork: Multi-Agent Systems

Why Use a Team of Agents?

For complex problems, a single agent isn't enough.

- **Divide and Conquer:** Break down a massive task into smaller, manageable sub-tasks.
- **Specialization:** Each agent can be an expert in a specific domain (e.g., a "research agent," a "writing agent," a "coding agent").
- **Modularity & Scalability:** Easier to build, debug, and upgrade individual specialist agents than one giant, monolithic agent.

The key to success?

Effective Communication

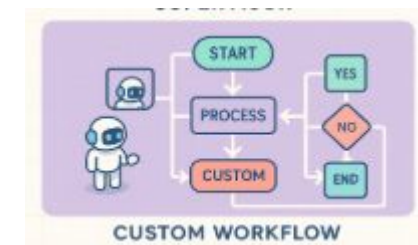
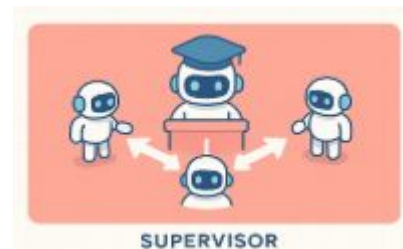
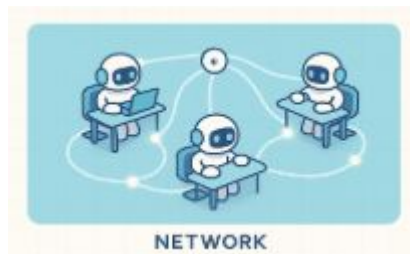


Multi-Agent Communication Architectures

How Agents Collaborate: Communication Architectures

The structure of communication defines how the team works.

- **Network:** An open-flow model where any agent can talk to any other agent. It's flexible but can be chaotic.
- **Supervisor:** A "manager" agent directs the workflow, telling other agents what to do and when. This provides control and order.
- **Hierarchical:** A multi-level management structure. A top-level manager oversees sub-managers, who in turn direct teams of agents. Ideal for very complex tasks.
- **Custom Workflow:** The flow is explicitly defined in a graph or sequence, offering a predictable and highly controlled process.



The "Handoff": How Information is Exchanged

Agents communicate by passing messages or "**payloads**."

- This process is often called a **Handoff**.
- The payload contains the necessary data for the next agent to act.
- **JSON (JavaScript Object Notation)** is the most common format for these payloads because it is:
 - **Structured:** Organizes data in key-value pairs.
 - **Lightweight:** Easy and fast to transmit.
 - **Human-Readable:** Simple for developers to debug.

```
{
  "source_agent": "Harvester-01",
  "target_agent": "Dryer-03",
  "task_id": "batch-789",
  "data": {
    "leaf_type": "Assam",
    "weight_kg": 50,
    "harvest_time": "2025-06-08T19:30:00Z"
  }
}
```

Analogy: The Chai Tea Factory

To understand communication, let's imagine a factory where each step is a specialized agent:

- 🌱 **Farmer Agent:** Plants the tea.
- 🌾 **Harvester Agent:** Collects the leaves.
- 🔥 **Dryer Agent:** Dries the leaves.
- 🥄 **Mixer Agent:** Blends leaves with spices.
- 🍷 **Bottler Agent:** Packages the final product.

How they talk to each other defines the workflow.



Network vs. Supervisor

Communication Style 1: Network (Direct)



Everyone can talk to everyone.

- **How it Works:** An agent finishes its task and directly tells the next agent in the process to start.
- **Chai Factory Example:** The **Harvester Agent** finishes and says directly to the **Dryer Agent**, "The leaves are ready for you."
- **Best for:** Simple, linear workflows and small teams. Fast and flexible.

Communication Style 2: Supervisor (Centralized)



One agent is the manager. All communication goes through them.

- **How it Works:** Agents report their status to a central **Supervisor Agent**, which then gives out the next command.
- **Chai Factory Example:** The **Harvester Agent** tells the **Factory Manager**, "I'm done." The Manager then instructs the **Dryer Agent**, "Start drying the new batch."
- **Best for:** Tasks requiring tight control, order, and error handling.

Hierarchical vs. Event-Driven

Communication Style 3: Hierarchical

A chain of command with multiple levels of management.



- **How it Works:** A top-level supervisor manages sub-supervisors, who in turn manage teams of agents.
- **Chai Factory Example:** A **General Manager** oversees a **Production Manager** (for farming, harvesting, drying) and a **Logistics Manager** (for bottling, delivery).
- **Best for:** Extremely complex, large-scale problems that can be broken down into distinct domains.

Communication Style 4: Event-Driven (Decoupled)

Agents don't give orders; they react to events.

- **How it Works:** Imagine a shared digital bulletin board. When an agent finishes a task, it posts a message (an "event"). Other agents are "listening" for messages relevant to them and act automatically when they see one.
- **Chai Factory Example:** The **Mixer Agent** posts a message: "Batch #123 is mixed." The **Bottler Agent**, which is subscribed to this type of message, sees it and starts its work without being told directly.
- **Best for:** Scalable, efficient, and resilient systems. Agents are not dependent on each other directly.

Part 2 Inside the Mind of AI Agents – Practice & workflow in n8n

- **The Fundamental Structure of an AI Agent in n8n**
- **A Real Case: Building Agent with n8n**
- **More Intros: Multi-Agent System**
- **RAG:When Agent dealing with files**

WORKFLOW AUTOMATION

WORKFLOW NODES

TRIGGER

PROCESSING

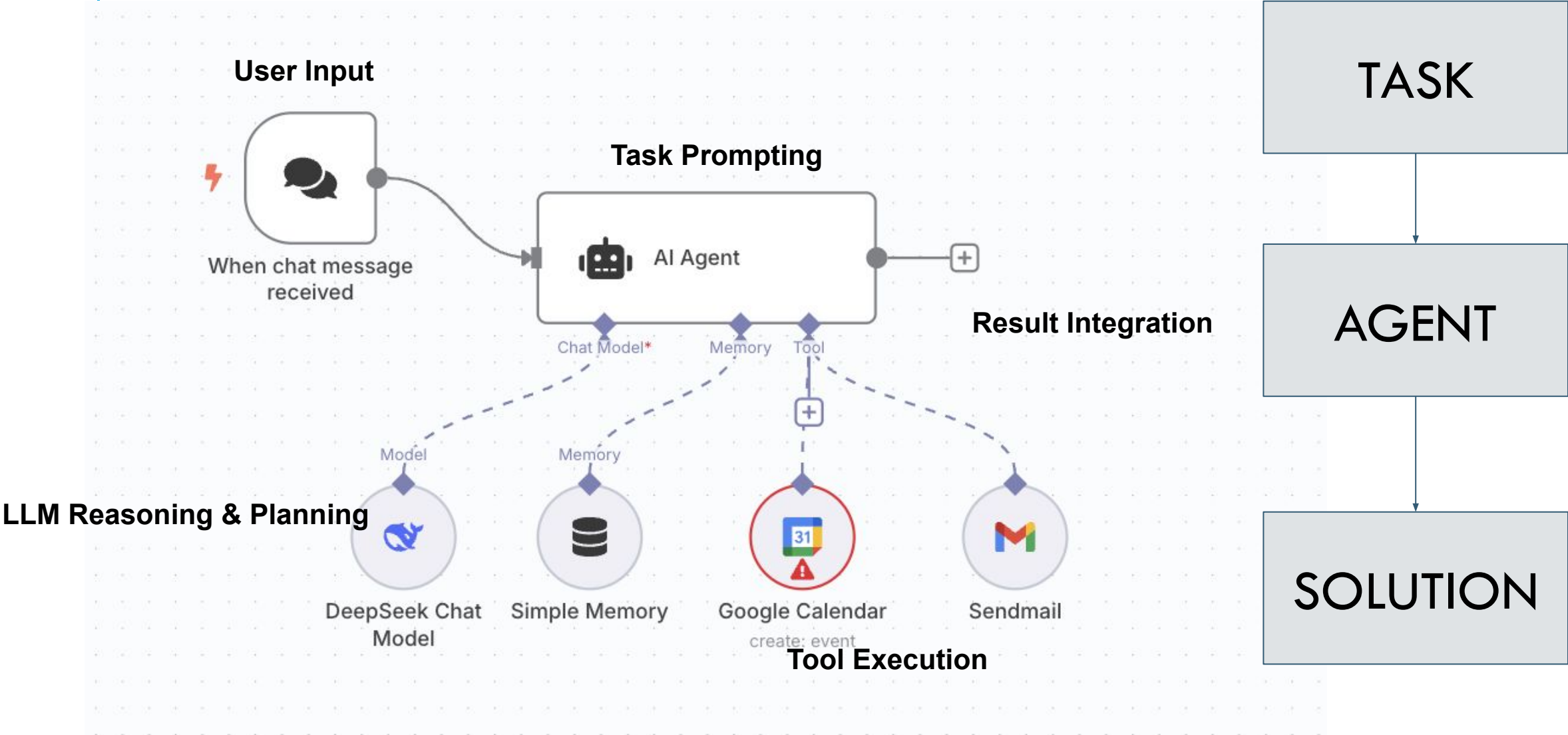
ACTION

- . Forms
- . File system
- . Schedule
- . Webhooks
- . External Apps
- . Chat message
- . Email received
- . SMS received
-

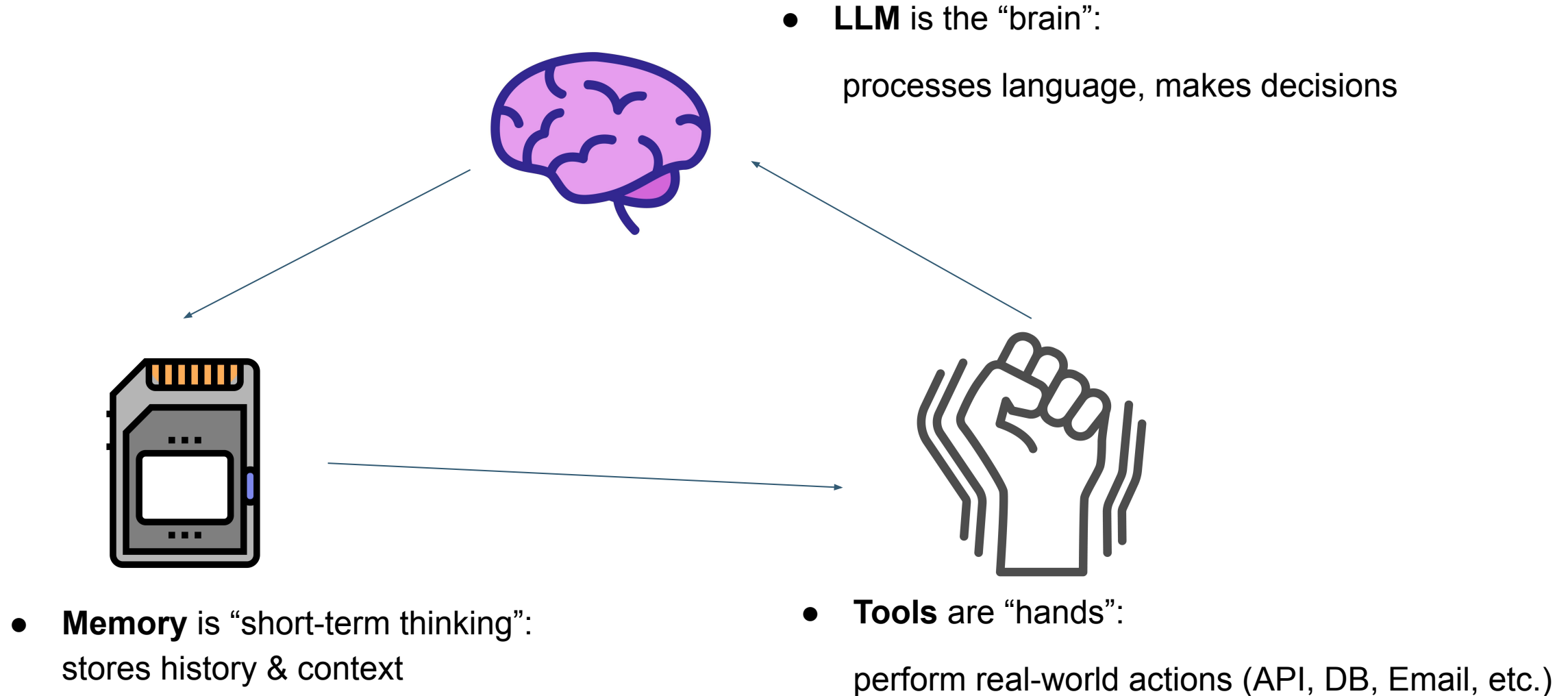
- . Sort
- . Filter
- . If Then
- . Transform
- . Enrich
- . AI Agents
- . AI Summarize
- . Transcribe Audio
- . Data validation
- . Math operations
-

- . Send Email
- . Post on Social Media
- . Save files
- . Create tickets
- . Make API calls
- . Update Database
- . Get Calendar Events
- . Generate AI Images / Voice / Video
-

A Simple Task Process AI agent in n8n

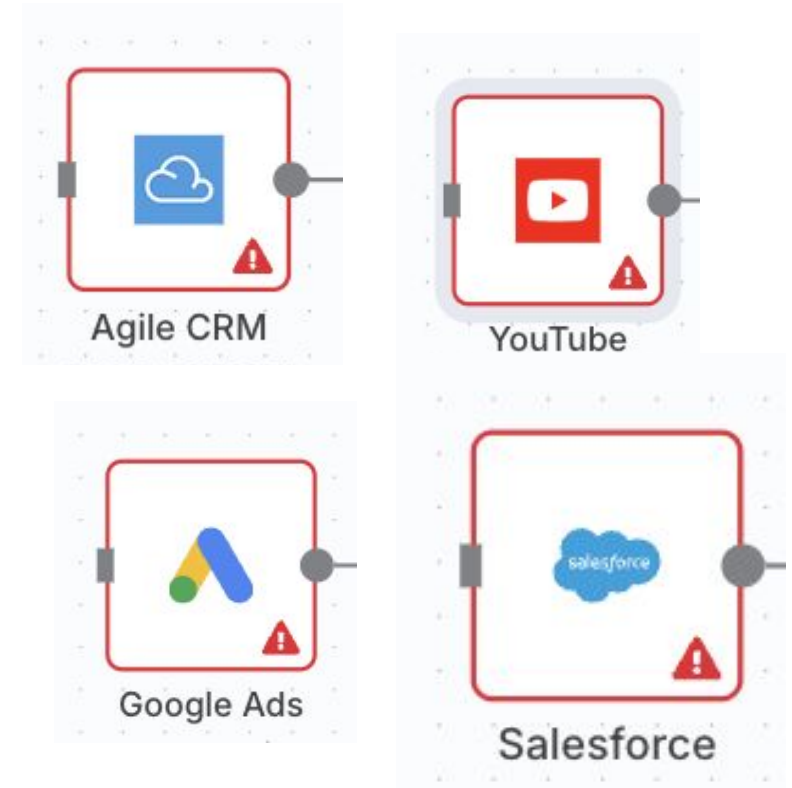


LLM + Tool + Memory: the components of AI Agent



What AI Agents Do

1. Personal Assistant
2. Social Media Manager
3. Customer Support
4. Research Assistant
5. Travel Planner



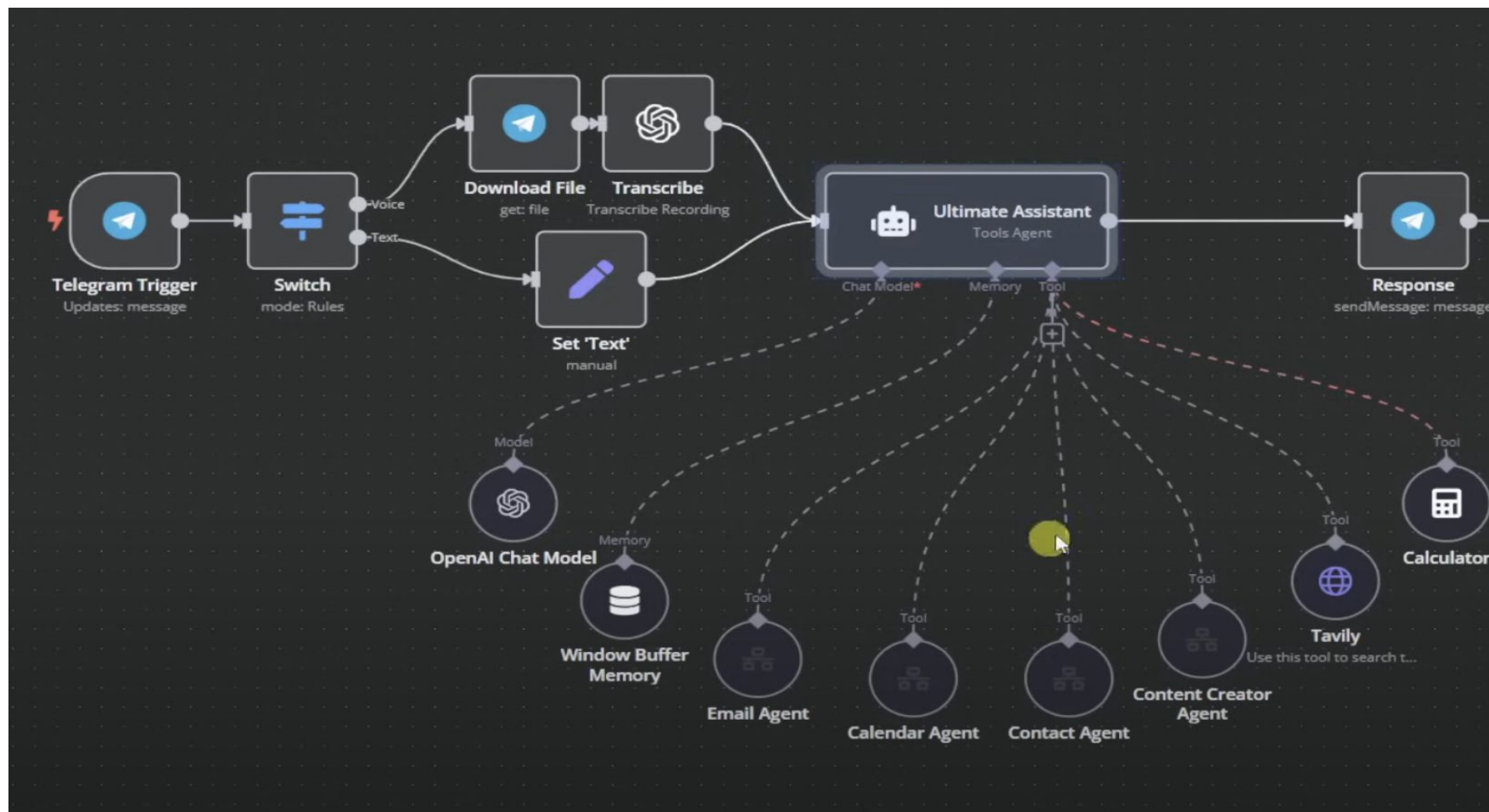
and more...

Case Demo: Building a Simple Agent with n8n



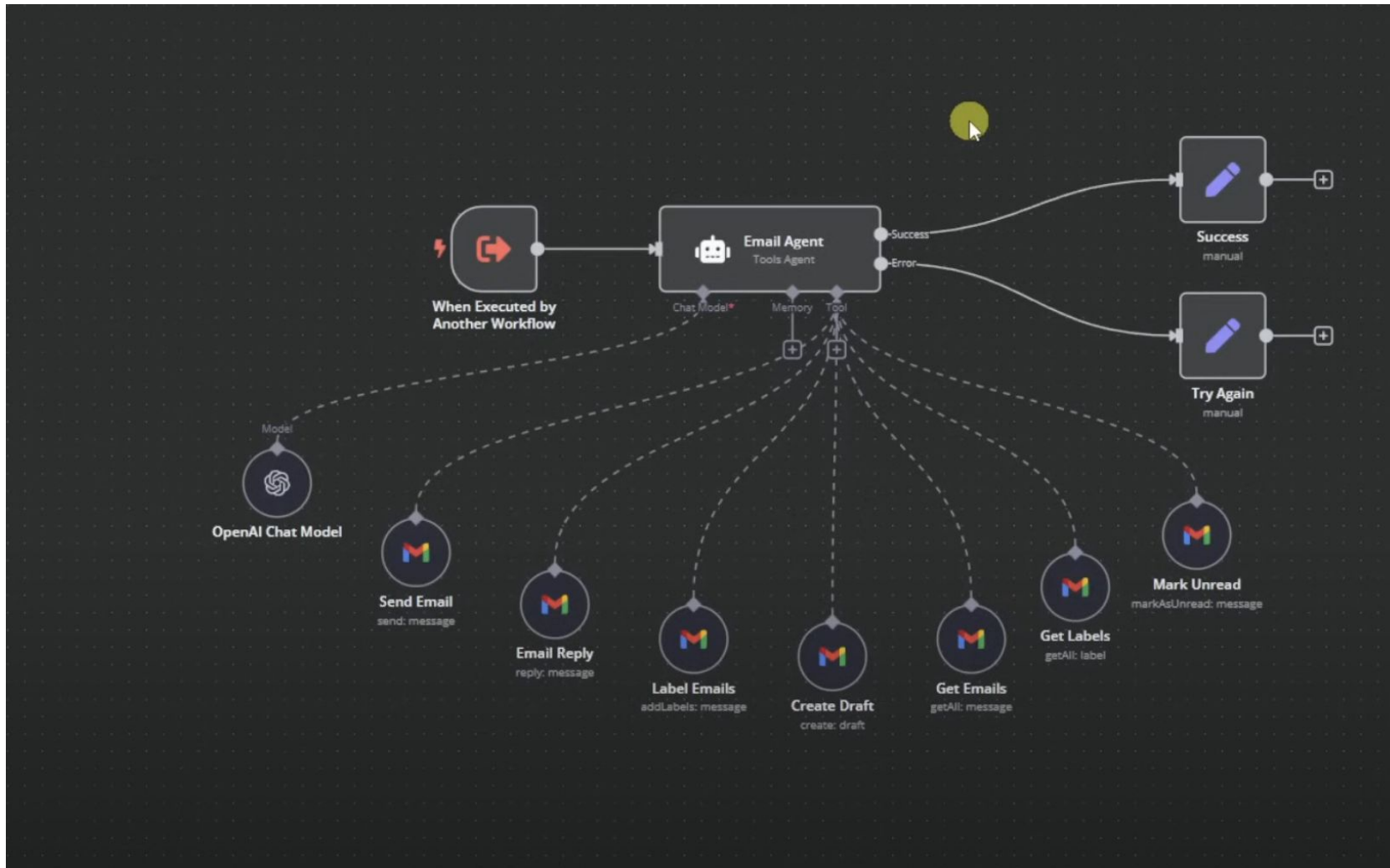
More types of System in Application

Multi-Agent System — Supervisory Agent



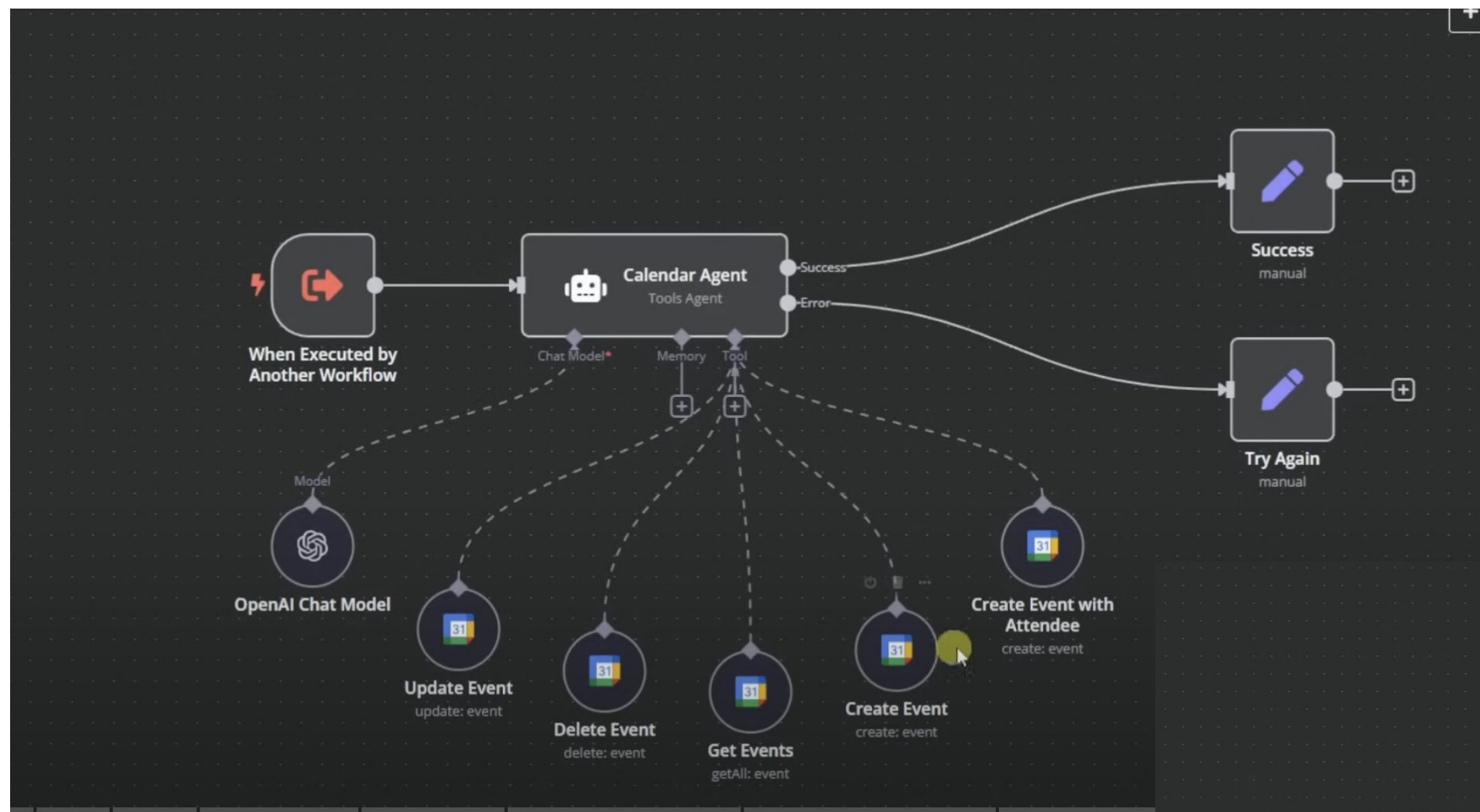
More types of System in Application

Multi-Agent System — Sub Email Agent



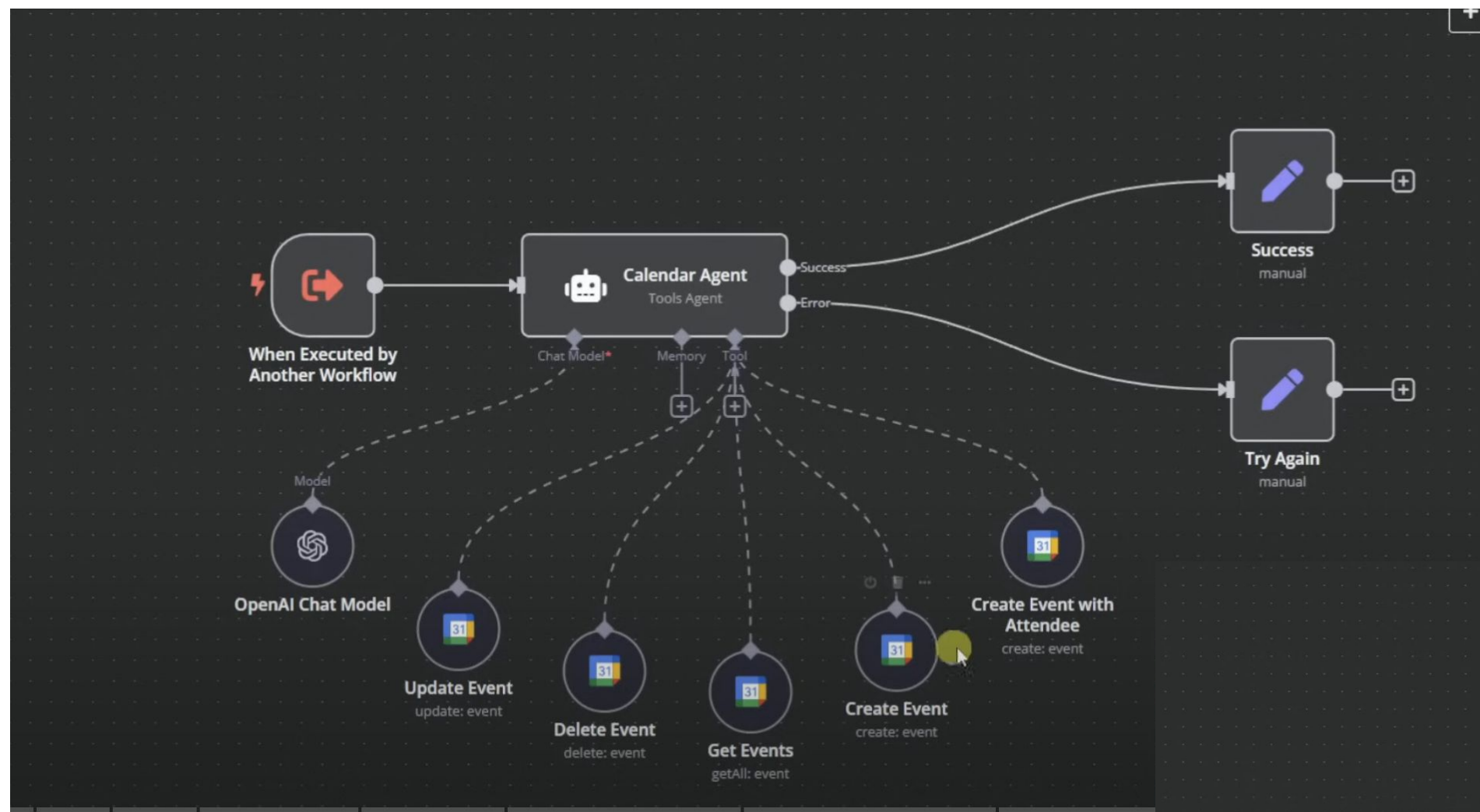
More types of System in Application

Multi-Agent System — Sub Calendar Agent



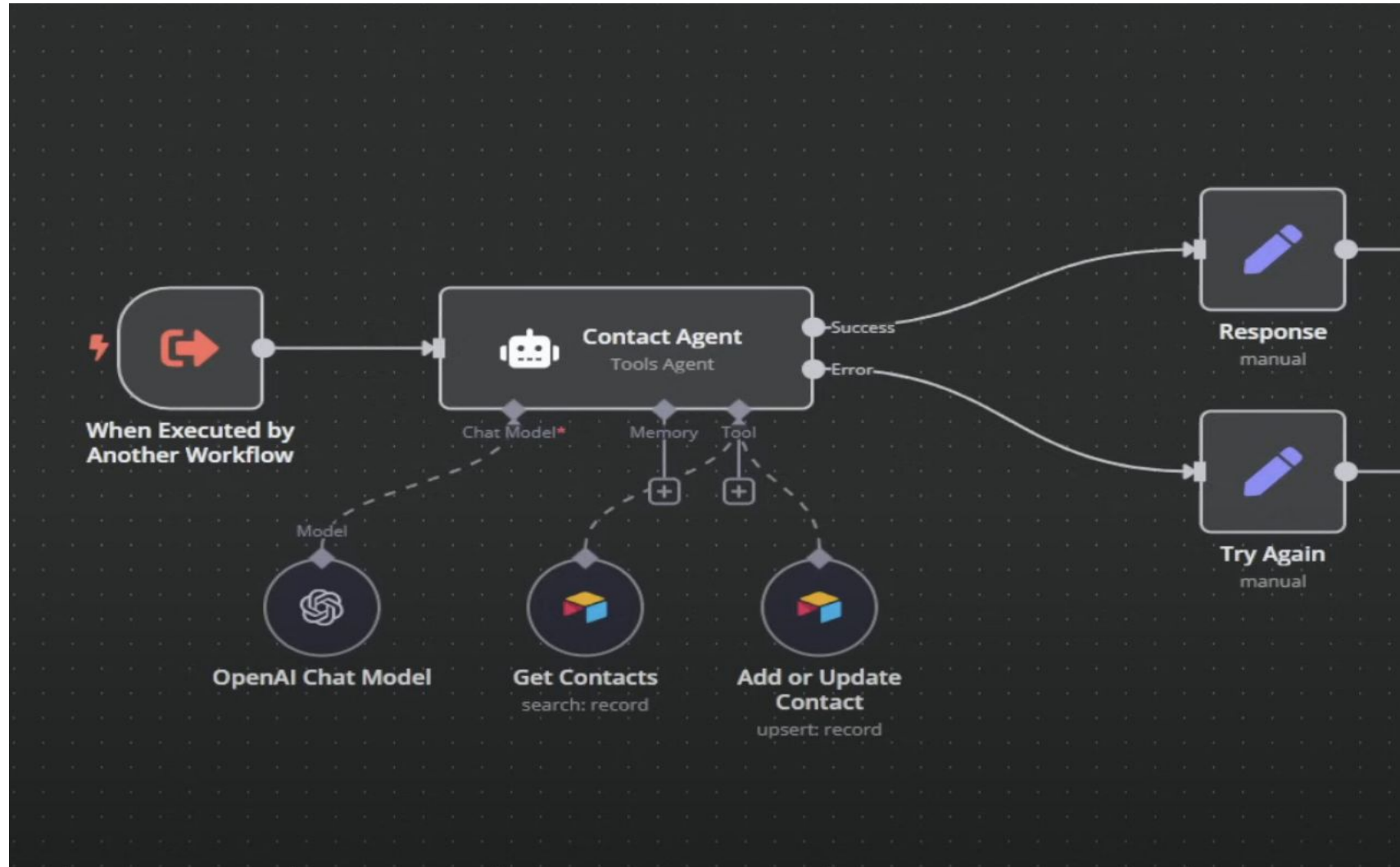
More types of System in Application

Multi-Agent System — Sub Calendar Agent



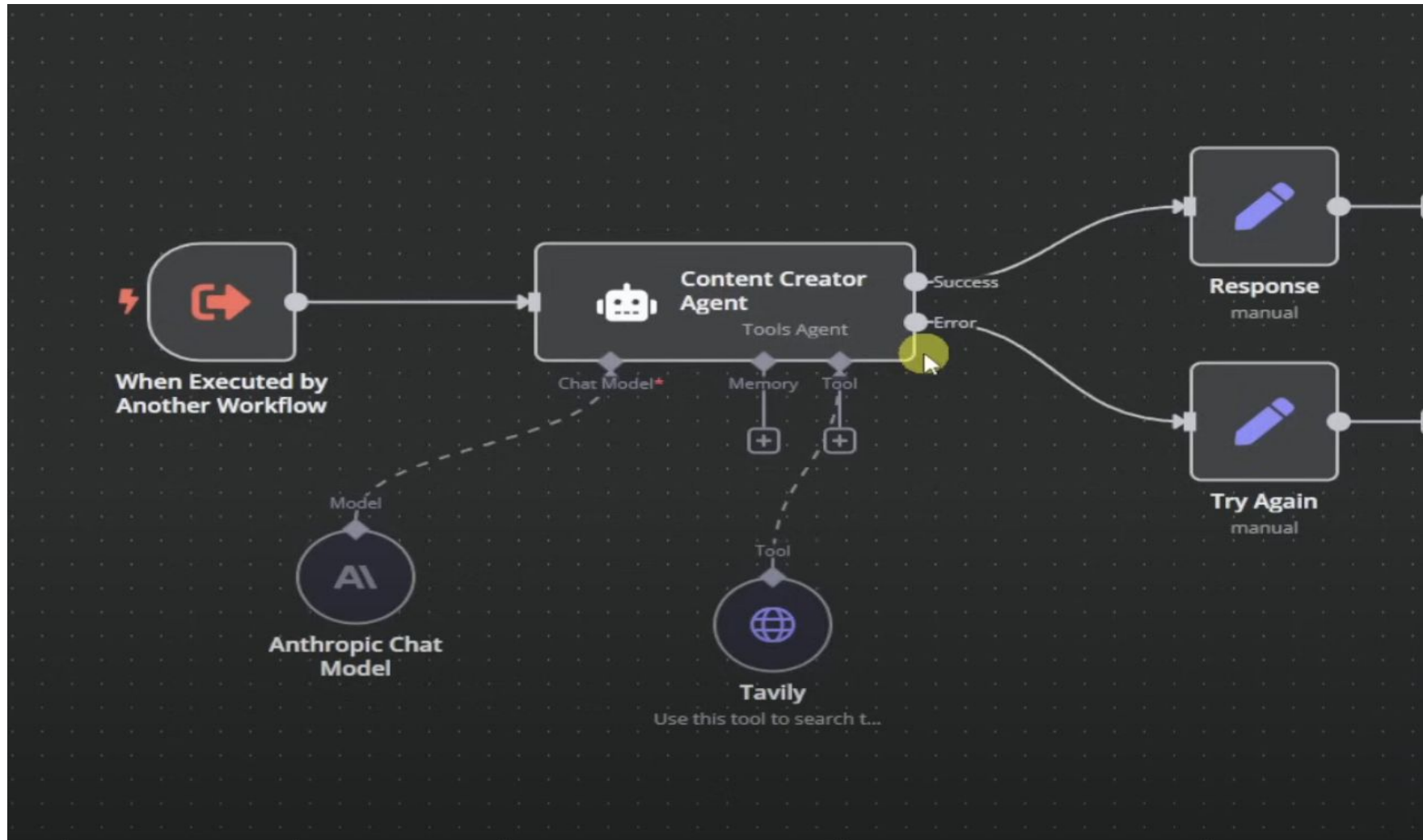
More types of System in Application

Multi-Agent System — Sub Contact Agent

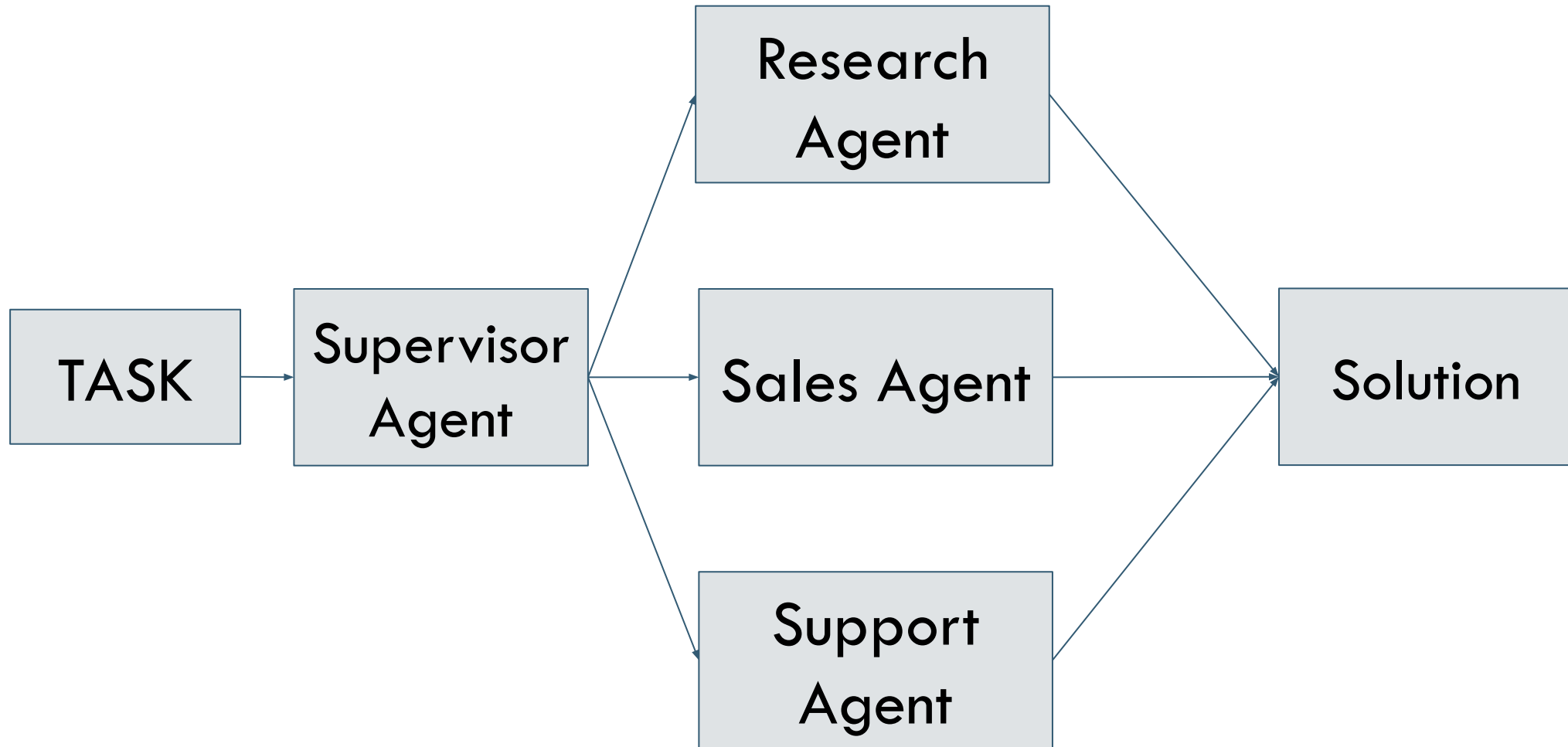


More types of System in Application

Multi-Agent System — Sub Content Creator Agent

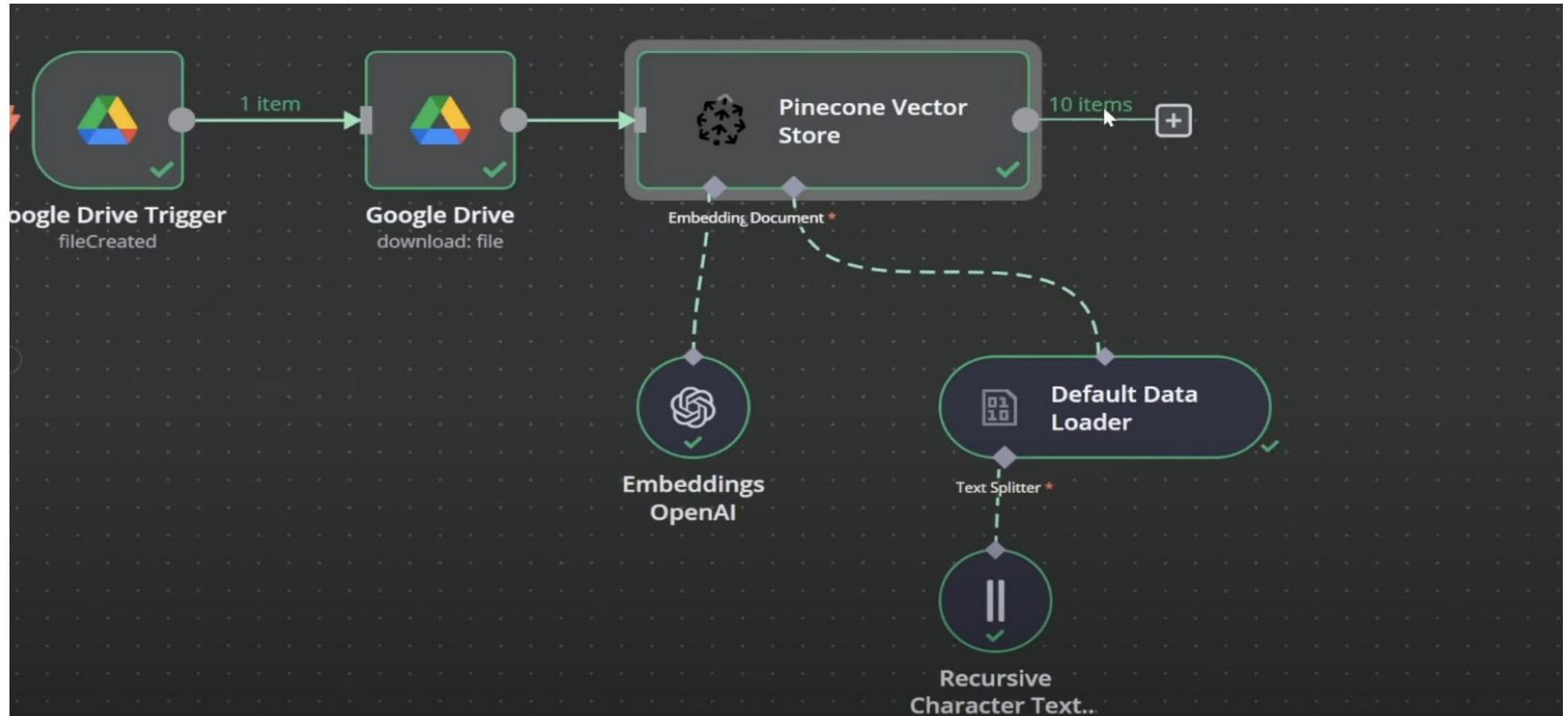


More types of System in Application



More types of System in Application

Vector Store As a Tool in AI Agent



More types of System in Application

Vector Store As a Tool in AI Agent

The screenshot displays the n8n Chat Window (AI Agent) interface. The chat window is titled "Chat Window (AI Agent)" and shows a conversation with an AI agent named "Oak & Barrel Assistant". The chat history includes:

- User: Hello
- AI: Hi there! How can I assist you today?
- User: What are your current specials?
- AI: This week at Oak & Barrel, we have some great specials for you! We're offering 50% off all steaks and happy hour from 4 PM to 6 PM. Come on in and enjoy!

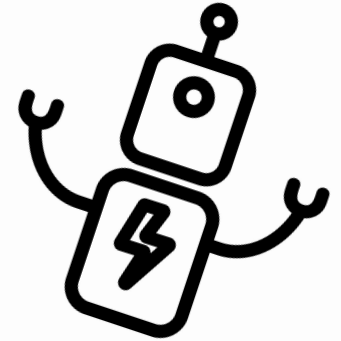
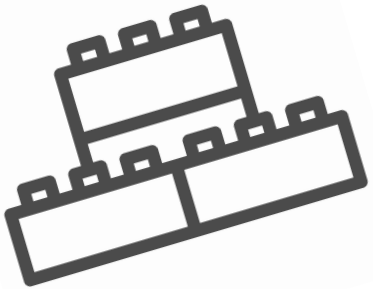
The interface also features a "Log (for last message)" section on the right, which shows the "Window Buffer Memory" and the "Input" data. The input data is a JSON object:

```
{  "action": "loadMemoryVariables",  "values": {    "input": "What are your current specials?",    "system_message": "You are a customer support agent for a restaurant called Oak & Barrel.\nYour name is Max.\nYour tone should be friendly and helpful.\nWhen asked questions about the restaurant,"  }}
```

The interface includes a sidebar with navigation options: Home, Templates, Variables, All execution, and Help. The top bar shows the n8n logo, the assistant name "Oak & Barrel Assistant", a "+ Add tag" button, and status indicators for "Inactive", "Share", and "Save".



Build the Simplest Things that Works



References

GitHub. (2025, February 6). *What are AI agents?* GitHub. <https://github.com/resources/articles/ai/what-are-ai-agents>

Kanerika, & Kanerika. (2025, May 8). *AI Agent Architecture: Breaking down the framework of autonomous systems*. Kanerika. <https://kanerika.com/blogs/ai-agent-architecture/>

Fuertes, R. A. (2025, June 2). *SmythOS - What is Agent Architecture? Understanding Its Importance in AI Systems*. SmythOS.

<https://smythos.com/developers/agent-development/what-is-an-agent-architecture/>

Orq.ai. (n.d.). *AI Agent Architecture: Core Principles & Tools in 2025 | Generative AI Collaboration Platform*. <https://orq.ai/blog/ai-agent-architecture>

THANKS

Group 3

